Property Rental & Listing Portal

GROUP 8 | N. Dharma, L. Siddartha, P. Dhanush, Prashanth | Team size: 4 | Duration: 7 days

Backend	Frontend	Database	Auth	Deployment
Flask (REST API)	React + Vite	PostgreSQL + SQLAlchemy	JWT (access + refresh)	Docker Compose

1. Problem Statement

Owners and agents list properties with photos, seekers search with many filters and request visits, and an admin verifies every listing before it appears in public search.

2. Roles & Permissions

Role	What they can do
Admin	Verifies or rejects listings, manages amenities and localities, handles reports.
Owner / Agent	Creates listings, uploads photos, answers enquiries, approves visit requests.
Seeker	Searches, shortlists, enquires and requests visits.

3. Data Model

- **User** - credentials, role, phone, verified flag.
- **Property** - owner, title, type (flat / house / PG / commercial), city, locality, rent, deposit, BHK, area, furnishing, available_from, status (draft / pending / verified / rejected / rented), view count.
- **PropertyImage** - property, stored file path, is_cover.
- **Amenity** + **property_amenity** - many-to-many.
- **Enquiry** + **EnquiryMessage** - seeker, property, thread of messages.
- **VisitRequest** - property, seeker, requested slot, status (requested / approved / rejected / completed).
- **Shortlist** and **SavedSearch** - per seeker.

4. Functional Requirements

- Listing CRUD with multi-image upload stored on a Docker volume and served by Flask.
- Amenity selection as a many-to-many relation.
- Admin verification queue with a rejection reason sent back to the owner.
- Public search: city, locality, rent range, BHK, property type, furnishing, available-from date, amenities, plus sort by rent or newest and pagination.
- Shortlist, enquiry thread between seeker and owner, visit scheduling with owner approval.
- Saved searches a seeker can re-run with one click.
- Owner analytics: views, enquiries, visit requests and conversion per listing.

5. Core Technical Challenge

One composable server-side search query. Filters are combined dynamically into a single SQLAlchemy query with joins for amenities, indexes on city/locality/rent, `LIMIT/OFFSET` pagination and a total count - never fetch all rows and filter in Python or in React. Unverified, rejected and rented listings are excluded from public search automatically, while the owner still sees their own. Uploads are validated for extension and size, renamed server-side, and a seeker can never overwrite another owner's files.

6. React Screens

- Search page: filter sidebar, result grid, sort control, pagination.
- Property detail with image gallery, amenities and an enquiry form.
- Shortlist and saved searches.
- Owner dashboard: my listings, image manager, enquiries, visit requests, analytics.
- Admin verification queue.

7. Authentication & Authorisation Specification

- Passwords hashed with bcrypt - plain text passwords must never touch the database.
- Endpoints: `POST /auth/register`, `/auth/login`, `/auth/refresh`, `/auth/logout`, `GET /auth/me`.
- The access token carries a `role` claim; a custom decorator `@role_required("admin")` guards privileged routes on top of `@jwt_required()`.
- Refresh rotation: a refresh call issues a new access token; logout adds the token JTI to a blocklist so it cannot be reused.
- React side: Axios request interceptor attaches the token, response interceptor catches 401, calls refresh once, retries the original request, and logs the user out if refresh also fails.
- Route protection in React by role - a seeker must never be able to open an admin screen, even by typing the URL.

8. Docker Specification

- `backend/Dockerfile` - Python slim base, requirements installed first for layer caching, served by `gunicorn` (not the Flask dev server).
- `frontend/Dockerfile` - multi-stage: Node stage runs `npm run build`, nginx stage serves the static build and proxies `/api` to the backend.
- `docker-compose.yml` - services `api`, `web`, `db` on one network, a named volume for database data, a healthcheck on `db`, and `depends_on` so the API waits for it.
- All secrets (DB URL, JWT secret key) come from `.env`; `.env.example` is committed, `.env` is not.
- Acceptance test: on a clean machine, `git clone` then `docker compose up --build` gives a working app with seeded demo data.

9. Phase-wise Execution Plan

Phase 1 - Day 1: Foundation

- Finalise the data model and draw the ER diagram / collection diagram before writing code.
- Repo and folder structure: `backend/`, `frontend/`, `docker-compose.yml`, `README.md`.
- Docker skeleton up early - database container running and reachable from the API container on day one.
- User model + full JWT auth flow + role seeding. Do not move on until login and refresh work in Postman.

Phase 2 - Day 2 to 3: Core API

- Every core CRUD endpoint with request validation and clear error responses (400 / 401 / 403 / 404 / 409).
- Role guards on every write route; ownership checks so a user can only touch their own records.
- Pagination, filtering and sorting on all list endpoints - never return the whole table.
- Test every endpoint in Postman and save the collection before you write a single React component.

Phase 3 - Day 3 to 5: React Application

- Routing, auth context, protected route wrapper, login and register screens.
- List, detail and create/edit screens for the main entities with client-side form validation.
- Loading, empty and error states on every screen - no blank page while data is fetching.
- Shared Axios instance; no API URL hardcoded in components.

Phase 4 - Day 5 to 6: Core Business Logic

- Implement the core rule described above - this is what separates this project from a CRUD demo.
- Dashboard and aggregation endpoints computed in the database layer, not in JavaScript.
- Search and filters wired end to end.

Phase 5 - Day 6 to 8: Harden and Ship

- Full dockerization verified from a clean clone; fix anything that only worked on your laptop.
- Seed script that creates demo users for each role plus realistic sample data.
- README: architecture overview, setup steps, environment variables, API endpoint table, demo credentials.
- Postman collection, bug fixing pass, and a 5-minute demo recording walking through each role.

10. Work Split

Owner	Responsibility
Member 1	Auth, roles, users, amenities, localities.
Member 2	Listings CRUD, image upload and serving, verification workflow.
Member 3	Search query builder, shortlist, saved searches, enquiries, visit requests, analytics.
Member 4	Entire React application for all three roles.

Every member must commit backend code and review at least one pull request - no single member owns the whole repository.

11. Deliverables

- GitHub repository containing both applications with a readable commit history from every member.
- README.md with setup instructions, environment variables and an API endpoint table.
- ER diagram (SQL) or collection diagram (MongoDB).
- Seed script with demo data and demo credentials per role.
- Postman collection covering every endpoint.
- Working `docker compose up --build` on a clean machine.
- 5-minute demo video.

12. Evaluation

Area	Weight	What is checked
Functionality	30%	All listed features work end to end without manual database edits
API design & JWT	20%	Correct status codes, validation, role guards, refresh and blocklist working
Core business logic	20%	The project-specific core rule is correctly and safely implemented
React frontend	15%	Component structure, state handling, validation, loading and error states
Docker & docs	15%	One-command startup, clean compose file, complete README and seed data

13. Stretch Goals (only after everything above works)

- Map view using listing latitude and longitude.
- Similar-properties block on the detail page.